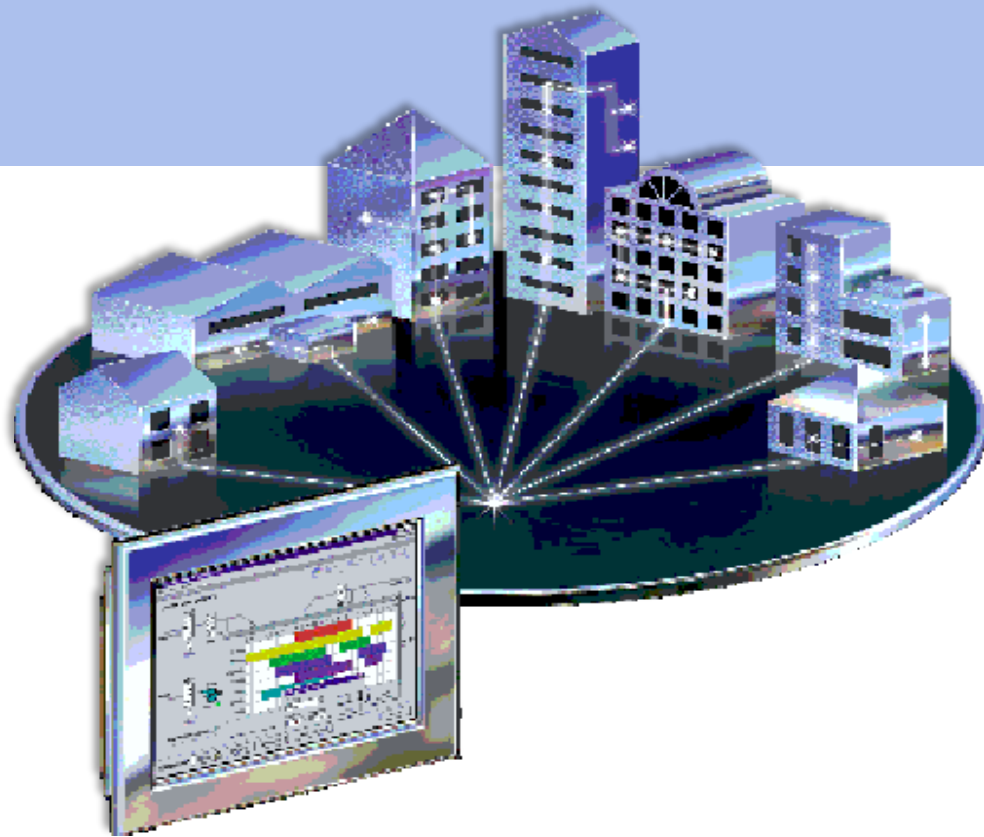


INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Práctica 7:



APLICACIÓN WEB PROGRESIVA

SISTEMAS DISTRIBUIDOS

Profesor || Chadwick Carreto Arellano
Grupo || 7CM1
Alumno || Areli Alejandra Guevara Badillo

Abril 29, 2026

FUNDAMENTOS TEÓRICOS DE LAS PROGRESSIVE WEB APPS (PWA)

Las Progressive Web Apps (PWA) constituyen un salto evolutivo en el desarrollo de software moderno. Su objetivo central es erradicar la brecha de experiencia de usuario que tradicionalmente ha separado a las páginas web convencionales de las aplicaciones nativas (móviles o de escritorio).

A través del uso de tecnologías web estandarizadas, una PWA ofrece una experiencia inmersiva idéntica a la de una aplicación nativa, pero con la ventaja de ser distribuida directamente mediante el navegador, eliminando la fricción de depender de tiendas de aplicaciones de terceros (como App Store o Google Play).

El término "Progresivo" se fundamenta en el principio de **Mejora Progresiva** (*Progressive Enhancement*): la aplicación está diseñada para funcionar con las características básicas en cualquier navegador, pero su experiencia y capacidades se enriquecen progresivamente si el dispositivo o navegador del usuario soporta tecnologías modernas.

Pilares Tecnológicos Fundamentales

La arquitectura de cualquier PWA se sostiene sobre dos componentes tecnológicos esenciales:

A. El Service Worker (El Motor Lógico y Offline)

Es el componente técnico más crítico de la arquitectura. Consiste en un script de JavaScript que el navegador ejecuta en segundo plano, en un hilo de procesamiento completamente independiente de la página web principal (el DOM). Sus funciones principales incluyen:

Proxy de Red Programable: Se interpone entre la aplicación y la red, lo que le permite interceptar, modificar y gestionar absolutamente todas las peticiones (métodos *fetch*) que realiza la aplicación.

Gestión Inteligente de Caché: Evalúa y decide cómo servir los recursos. Puede priorizar la red (*Network First*), priorizar el almacenamiento local (*Cache First*), o utilizar estrategias híbridas (*Stale-While-Revalidate*).

Capacidad Offline: Al almacenar recursos estructurales (HTML, CSS, JS e imágenes) en la memoria caché, garantiza que la aplicación siga siendo funcional incluso ante la pérdida total de conexión a internet.

Procesos en Segundo Plano: Actúa como puente para la integración de características nativas avanzadas, como la recepción de Notificaciones Push y la sincronización de datos en segundo plano.

B. El Web App Manifest (El Archivo de Identidad)

Se trata de un archivo de configuración en formato JSON (.webmanifest) cuyo propósito es describir la aplicación ante el sistema operativo, permitiendo que el dispositivo la trate como un software nativo. Define metadatos cruciales como:

Identidad: Nombre completo (name) y el nombre corto (short_name) aparecerá debajo del ícono.

Recursos Gráficos: Un arreglo de íconos (icons) en múltiples resoluciones para adaptarse a pantallas de inicio y selectores de tareas.

Comportamiento Visual: Atributos como `display` (usualmente configurado en *standalone* para ocultar la barra de direcciones del navegador) y `theme_color` (para teñir la barra de estado del celular).

Punto de Entrada: La ruta principal (`start_url`) que el sistema debe cargar al abrir la aplicación.

Características Clave de una PWA

Cuando la implementación del *Manifest* y el *Service Worker* es correcta, la aplicación web hereda un conjunto de atributos distintivos:

1. **Instalabilidad:** El navegador detecta automáticamente los pilares de la PWA y ofrece al usuario la opción nativa de "Añadir a la pantalla de inicio", instalándola junto al resto de sus aplicaciones.
2. **Resiliencia de Conectividad:** La aplicación carga de manera casi instantánea y mantiene su operatividad en entornos sin conexión o con redes intermitentes.
3. **Seguridad Estricta:** Las PWA exigen obligatoriamente el uso del protocolo **HTTPS**. Debido a que el *Service Worker* tiene el poder de interceptar peticiones de red, el cifrado es un requisito innegociable para prevenir ataques de intermediarios (*Man-in-the-Middle*).
4. **Retención de Usuarios (*Re-engageable*):** Al soportar Notificaciones Push, la PWA puede emitir alertas proactivas al dispositivo del usuario, incluso si el navegador está cerrado, incrementando significativamente el uso frecuente.
5. **Descubrible y Enlazable:** A diferencia de las apps nativas en tiendas cerradas, una PWA sigue siendo parte de la web; por lo tanto, su contenido es indexable por motores de búsqueda (SEO) y la aplicación entera puede compartirse con un simple enlace (URL).

PLANTEAMIENTO DEL PROBLEMA

En el aprendizaje y dominio de la arquitectura de servicios web, el primer gran desafío es comprender de manera práctica la interacción entre un cliente y un servidor mediante el protocolo HTTP en un entorno "sin estado" (stateless). A menudo, los desarrolladores noveles luchan por asimilar el uso semántico de los verbos HTTP y la estructura estandarizada de las respuestas JSON.

Sin embargo, una vez asimilados estos conceptos en el servidor (backend), surge una segunda barrera de complejidad: **la integración real con una interfaz de usuario (frontend) y la distribución del software**. Las prácticas tradicionales suelen omitir cómo un cliente consume estas APIs de manera asíncrona, cómo se gestionan políticas de seguridad cruzada (CORS) y cómo se refleja visualmente la mutación de datos.

Por lo tanto, se requiere un escenario de práctica integral (Full-Stack) que permita no solo manipular un estado interno en el servidor mediante reglas simples, sino también consumir esa API desde un cliente web moderno. Este entorno debe proporcionar retroalimentación visual inmediata sobre cómo las peticiones alteran el estado, cumpliendo además con los estándares actuales de distribución web.

PROPUESTA DE SOLUCIÓN

Para abordar esta necesidad, se diseñó e implementó un **"API-gotchi"**: una mascota virtual cuyo estado vital (niveles de hambre, energía y felicidad) es gestionado exclusivamente a través de una API RESTful, pero que es consumido y renderizado mediante una interfaz gráfica construida como una **Progressive Web App (PWA)**.

Esta solución divide el sistema en dos capas arquitectónicas claras, concentrando el esfuerzo técnico en:

1. **Backend (Diseño de la API y Gestión de Estado):** Construido con **FastAPI**, administra el ciclo de vida del recurso en la memoria del servidor. Implementa métodos HTTP semánticos (GET para lectura, POST para mutación) y un riguroso manejo de excepciones para retornar códigos de estado (Status Codes) coherentes cuando se violan las reglas de negocio. Además, el backend unifica el sistema sirviendo directamente los archivos estáticos de la PWA.
2. **Frontend (Consumo Asíncrono e Interfaz):** Una interfaz desarrollada con tecnologías web estándar (HTML, CSS y JS puro). Utiliza la API Fetch para comunicarse asíncronamente con el servidor. Emplea diseño responsivo y animaciones CSS (Keyframes) para reaccionar a los cambios de estado (ej. animaciones al comer o dormir).
3. **PWA (Instalabilidad y Experiencia Nativa):** Se integró un *Web App Manifest* para definir la identidad visual de la aplicación y un *Service Worker* para habilitar la política de caché, permitiendo que la interfaz sea instalable en dispositivo

Stack Tecnológico: Proyecto API-gotchi (Full-Stack PWA)

Categoría	Tecnología	Función en el Proyecto	¿Por qué elegirla?
Lenguaje Base (Backend)	Python (3.10+)	Lógica central, manejo del estado en diccionarios y evaluación de reglas de negocio.	Sintaxis limpia, excelente tipado (Type Hints) y curva de aprendizaje amigable.
Framework Web	FastAPI	Enrutamiento HTTP, validación de datos, manejo de CORS y alojamiento de archivos estáticos.	Rendimiento excepcional, moderno y diseñado específicamente para construir APIs RESTful.
Servidor ASGI	Uvicorn	Levantar la aplicación y escuchar peticiones HTTP de manera asíncrona.	Estándar de la industria para ejecutar aplicaciones modernas en Python con altísima velocidad.
Estructura y Estilos (Frontend)	HTML5 y CSS3	Construcción de la consola virtual, diseño responsivo (Media Queries) y animaciones de estado (Keyframes).	Permite crear interfaces complejas, atractivas y adaptables sin depender de frameworks pesados de diseño.
Lógica de Cliente	Vanilla JavaScript	Consumo asíncrono de la API mediante Fetch, manipulación del DOM y registro del Service Worker.	Evita la sobrecarga de usar librerías como React o Angular para un proyecto donde el control del DOM es directo.
Tecnologías PWA	Manifest & Service Worker	Definición de identidad (íconos, colores) y almacenamiento en caché para carga inmediata e instalación nativa.	Convierte una página web tradicional en una aplicación instalable en móviles y PC, mejorando la experiencia de usuario.
Entorno de Desarrollo	Visual Studio Code	Escritura de código, gestión de terminales y entorno virtual.	Ligero y versátil, ideal para manejar proyectos que combinan Python y tecnologías web simultáneamente.

DESARROLLO E IMPLEMENTACIÓN

La construcción del proyecto "API-gotchi" evolucionó de una API exclusivamente de backend a una arquitectura Full-Stack con capacidades de Progressive Web App (PWA). El desarrollo se dividió en fases secuenciales que abarcan la lógica del servidor, la interfaz gráfica y la integración de tecnologías web progresivas.

Arquitectura del Backend y Unificación de Servicios

El backend fue desarrollado utilizando FastAPI. Se implementaron rutas POST (/alimentar, /jugar, /dormir) que modifican el estado en memoria de la mascota aplicando reglas de validación estricta (ej. rechazar la acción de jugar si la energía es menor a 20 mediante un error HTTP 400). Se agregó un endpoint adicional, /reiniciar, para restablecer los valores vitales y manejar el escenario de "Game Over".

Para optimizar el despliegue y evitar problemas de políticas de mismo origen (CORS), se unificó el servicio utilizando la clase StaticFiles de FastAPI. Esto permitió que el mismo servidor que procesa la lógica de la API sirva los archivos estáticos de la interfaz web en la ruta raíz (/).

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles

app = FastAPI(title="API-gotchi PWA")

# ... [Definición de rutas de la API] ...

# Unificación: El backend sirve el frontend directamente
app.mount("/", StaticFiles(directory="frontend", html=True), name="frontend")
```

Interfaz de Usuario y Lógica Asíncrona (Frontend)

Se diseñó una interfaz responsiva (Responsive Design) inspirada en consolas de videojuegos retro. La interfaz se adapta dinámicamente: en dispositivos móviles ocupa el ancho de la pantalla, mientras que en resoluciones de escritorio (evaluadas mediante *Media Queries* en CSS) la consola se expande y reorganiza sus elementos para aprovechar el espacio.

En lugar de utilizar múltiples imágenes o recursos pesados, los estados de la mascota se representaron utilizando caracteres Unicode (Emojis) y animaciones CSS avanzadas (@keyframes).

La comunicación con el servidor se implementó con *Vanilla JavaScript* utilizando la API Fetch. Las peticiones asíncronas envían las acciones al servidor y, con base en la respuesta JSON, el DOM (Document Object Model) se actualiza de manera reactiva, activando animaciones de transición, cambiando el entorno (ej. oscureciendo la pantalla al dormir) y actualizando las barras de progreso.

Implementación de la Progressive Web App (PWA)

Para elevar el proyecto a los estándares web modernos y cerrar la brecha con las aplicaciones nativas, se implementó una arquitectura PWA. Esto requirió la configuración de dos componentes fundamentales: el Web App Manifest y el Service Worker.

Web App Manifest

Se creó el archivo manifest.json para definir la identidad de la aplicación ante el sistema operativo del dispositivo. Este archivo permite que navegadores móviles (como Chrome o Safari) identifiquen el software como instalable. Se configuró el modo standalone para que, al iniciarse desde la pantalla de inicio del teléfono, la aplicación se ejecute en pantalla completa, ocultando la barra de direcciones del navegador.



```
{
  "name": "API-gotchi Virtual",
  "short_name": "API-gotchi",
  "start_url": "./index.html",
  "display": "standalone",
  "background_color": "#ffeaa7",
  "theme_color": "#FFB6C1",
  "icons": [
    {
      "src": "icons/icon-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icons/icon-512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

Estrategia de Caché con Service Worker

El núcleo técnico de la PWA residió en la implementación del archivo sw.js (Service Worker). Este script de JavaScript actúa como un proxy de red que se ejecuta en segundo plano.

Durante la fase de instalación (install), el Service Worker abre una caché local en el dispositivo del usuario y almacena la "cáscara de la aplicación" (App Shell), la cual incluye el HTML, CSS y JavaScript estructural.

Durante la fase de intercepción de red (fetch), se implementó una estrategia híbrida. Si el navegador solicita archivos estáticos de la interfaz, el Service Worker los recupera instantáneamente de la caché, garantizando que la UI se renderice incluso sin conexión. Sin embargo, las peticiones dirigidas a la API (rutas dinámicas) se filtran para que siempre busquen la red, asegurando que el estado de la mascota no quede congelado por una versión almacenada en caché.

```
const CACHE_NAME = 'apigotchi-v1';
const urlsToCache = [
  './index.html',
  './style.css',
  './app.js',
  './manifest.json'
];

// Instalación: Almacenamiento de la App Shell
self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(CACHE_NAME)
      .then(cache => cache.addAll(urlsToCache))
  );
});

// Intercepción: Estrategia de Caché
self.addEventListener('fetch', event => {
  // Se ignora el almacenamiento en caché para peticiones de la API (backend)
  if (event.request.url.includes('127.0.0.1')) {
    return;
  }

  // Cache First con fallback a la red
  event.respondWith(
    caches.match(event.request)
      .then(response => {
        return response || fetch(event.request);
      })
  );
});
```

Ciclo de Registro

Finalmente, para activar estas funcionalidades, se agregó un bloque de validación en el cliente (app.js) que verifica si el navegador soporta Service Workers. De ser así, registra el script para habilitar las características progresivas de forma silenciosa al cargar la página.

```
if ('serviceWorker' in navigator) {
  window.addEventListener('load', () => {
    navigator.serviceWorker.register('sw.js')
      .then(reg => console.log('SW Registrado correctamente'))
      .catch(err => console.error('Fallo en registro de SW', err));
  });
}
```

Con estas implementaciones, el API-gotchi no solo procesa lógica backend mediante estándares REST, sino que se distribuye como un software moderno instalable, de carga inmediata y adaptable a cualquier dispositivo.

RESULTADOS

El desarrollo del proyecto concluyó exitosamente, logrando la evolución de una API estática a una aplicación Full-Stack completamente funcional y certificada bajo los estándares de las Progressive Web Apps (PWA). A continuación, se detallan los resultados obtenidos divididos en el comportamiento visual y el rendimiento técnico.

Interfaz Gráfica y Diseño Responsivo

Se logró renderizar una interfaz de usuario atractiva, basada en la estética de consolas retro, prescindiendo del uso de imágenes externas gracias al aprovechamiento del CSS moderno y caracteres Unicode (emojis).

El diseño demostró una correcta adaptabilidad (Responsive Design). En dispositivos móviles, la aplicación se ajusta a una vista vertical compacta, mientras que, al ejecutarse en resoluciones de escritorio, el diseño aprovecha el espacio disponible expandiendo el contenedor principal y escalando los elementos visuales, como el avatar de la mascota, para mejorar la experiencia inmersiva.

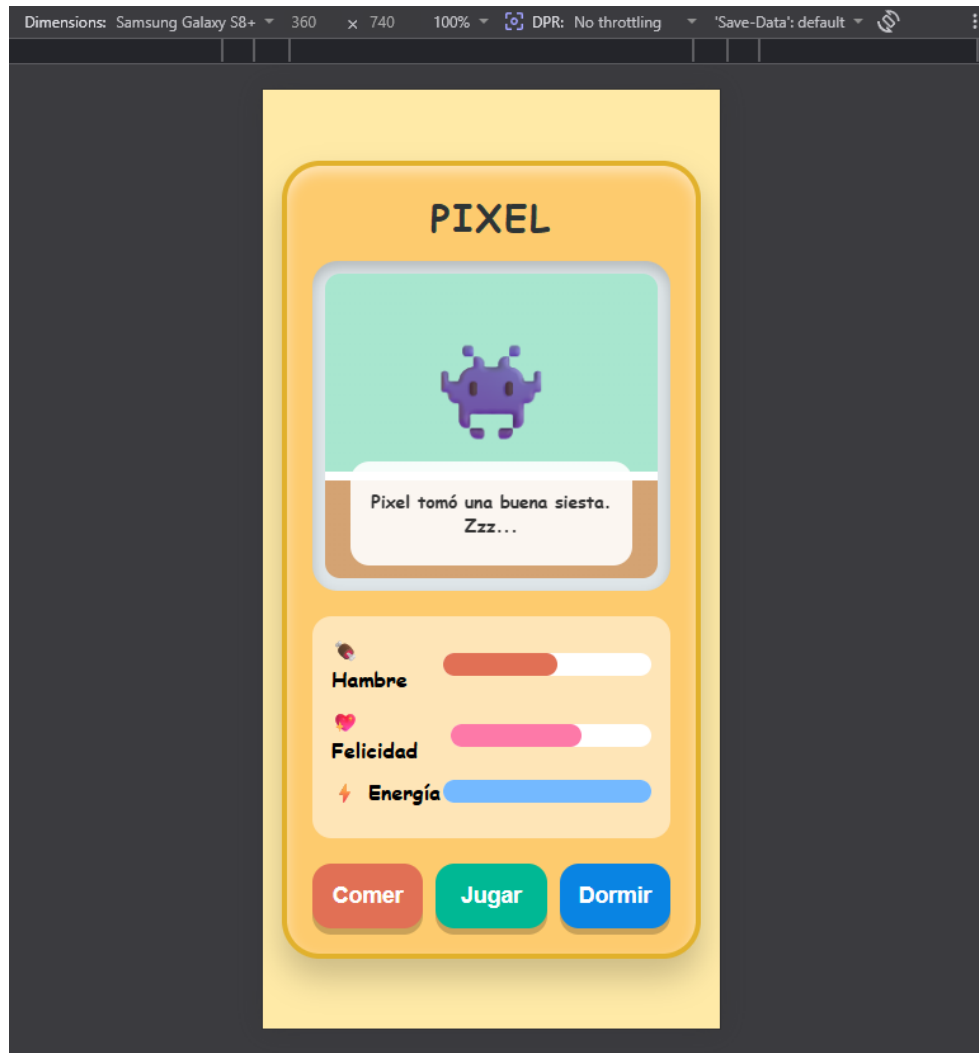


Figura 1: Vista móvil de la aplicación, mostrando las barras de estado y los controles principales.

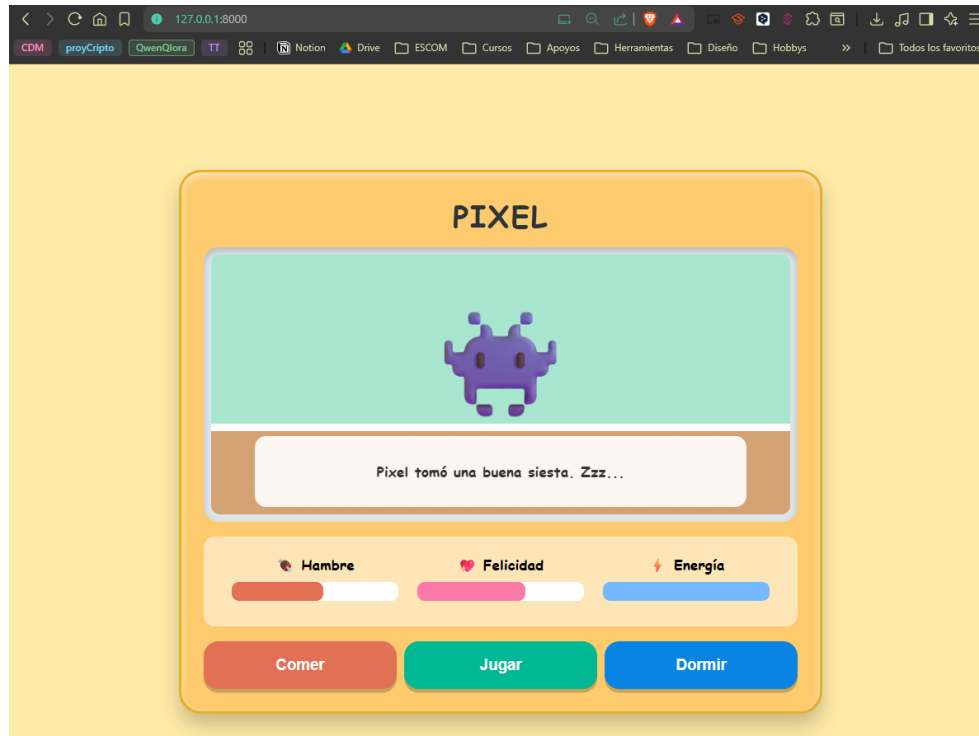


Figura 2: Vista de escritorio, demostrando la escalabilidad de la interfaz y la expansión de la consola.

Consumo Asíncrono y Reacción de Estados (Animaciones)

La integración entre el frontend y el backend mediante la API Fetch operó sin latencia perceptible. Al ejecutar acciones a través de los botones, el servidor procesa la mutación de estado y el cliente reacciona inmediatamente al JSON de respuesta.

Se comprobaron con éxito las transiciones visuales, incluyendo:

- El cambio de iluminación del escenario (efecto de día/noche) al invocar el endpoint /dormir.
- El cambio del avatar (de Alien a Tumba), la inhabilitación de los controles principales y la aparición dinámica del botón de "Revivir" cuando la API devuelve un estado "fallecido" por inactividad o exceso de hambre.

Práctica 7: Aplicación Web Progresiva



Figura 3: Reacción del entorno visual al interactuar con el endpoint de descanso (modo nocturno).

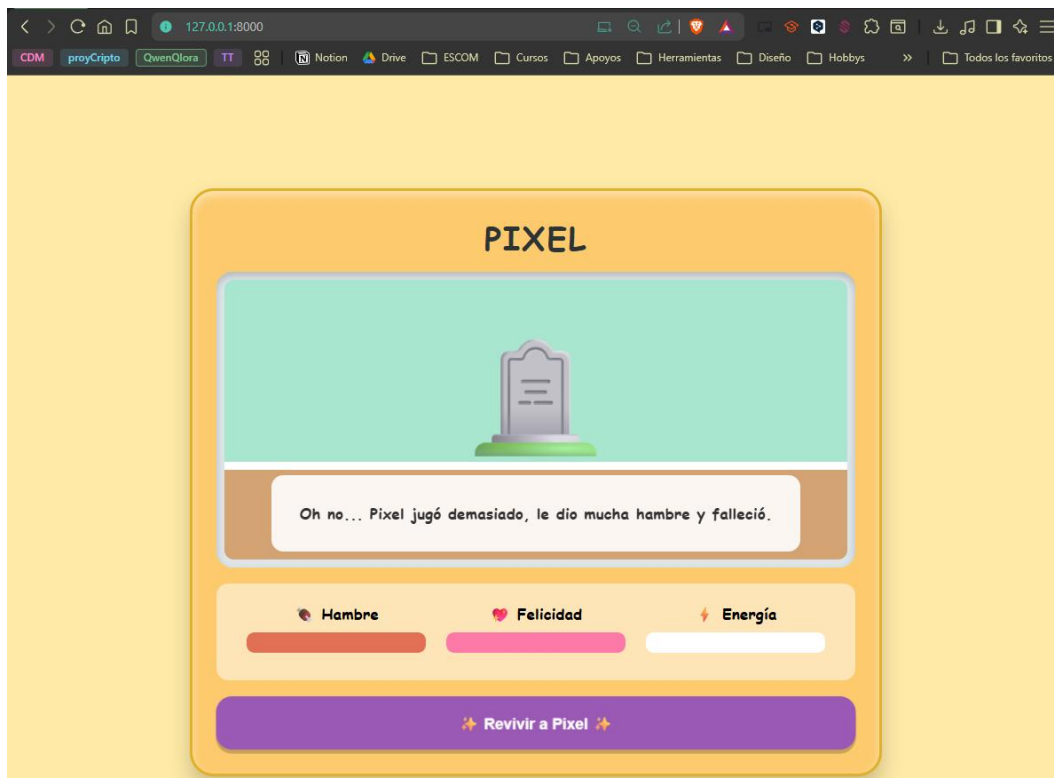


Figura 4: Manejo del estado crítico "Fallecido", donde la UI bloquea acciones inválidas y habilita la ruta de reinicio.

Capacidades PWA e Instalación Nativa

El resultado más destacado a nivel arquitectónico fue la exitosa validación de la aplicación como PWA. Al acceder a la ruta alojada por FastAPI, navegadores modernos como Google Chrome detectaron correctamente la presencia del manifest.json y el registro activo del sw.js (Service Worker).

Como resultado de esta configuración, el navegador habilitó nativamente la opción de instalación. Una vez instalada, la aplicación se ejecuta en una ventana independiente (modo standalone), eliminando la interfaz clásica del navegador (barra de búsqueda, pestañas), otorgando una experiencia indistinguible de una aplicación de escritorio o móvil tradicional.

Además, se verificó mediante las herramientas de desarrollador que la "App Shell" es almacenada en la caché del navegador, permitiendo que la interfaz gráfica cargue de forma instantánea en visitas posteriores.

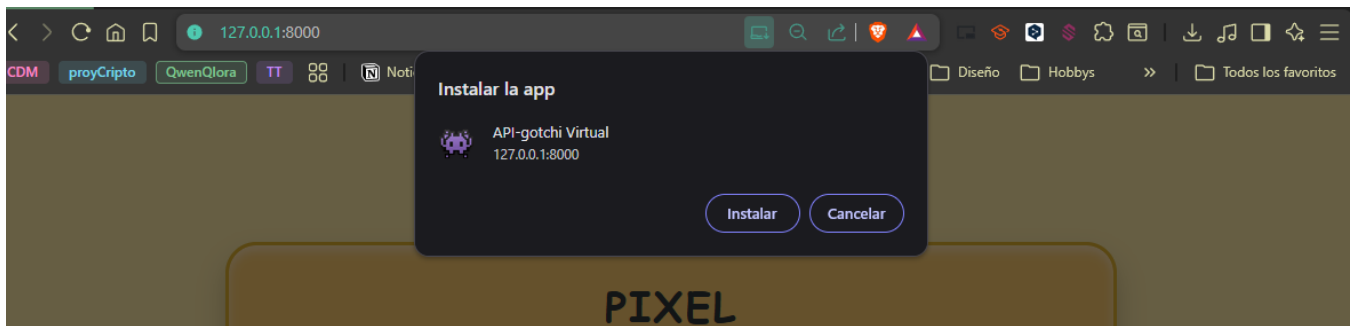


Figura 5: Detección automática del navegador de los estándares PWA, ofreciendo la instalación al usuario.



Figura 6: Aplicación ejecutándose en modo standalone como software de escritorio independiente.

CONCLUSIONES

La conceptualización, desarrollo e implementación del proyecto "API-gotchi" demostró ser un ejercicio académico y técnico altamente efectivo para consolidar los fundamentos de la arquitectura de sistemas modernos. A través de la evolución de este sistema —desde un simple servicio backend hasta una aplicación Full-Stack—, se alcanzaron las siguientes conclusiones clave:

1. **Robustez en el Diseño de APIs RESTful:** Se comprobó empíricamente que la delegación de responsabilidades en el servidor es vital. El uso semántico de los verbos HTTP (diferenciando operaciones de solo lectura como GET de operaciones de mutación como POST) y la emisión de excepciones estructuradas con los códigos de estado adecuados (familia HTTP 4xx) garantizan que las reglas de negocio sean respetadas, sin importar cómo se comporte el cliente.
2. **El Paradigma Cliente-Servidor y la Asincronía:** La integración del frontend evidenció el verdadero poder de una arquitectura desacoplada. El servidor (construido con FastAPI) ignora por completo la interfaz gráfica, limitándose a validar lógica y proveer datos estandarizados en JSON. Por su parte, el cliente web asume la responsabilidad exclusiva de la experiencia de usuario (UI/UX), utilizando peticiones asíncronas (Fetch) para mutar el DOM en tiempo real, demostrando un modelo altamente escalable y fácil de mantener.
3. **El Valor de las Progressive Web Apps (PWA):** La implementación de las tecnologías web progresivas (Web App Manifest y Service Worker) supuso un cambio de paradigma respecto a la distribución de software. Se constató que ya no es estrictamente necesario dominar lenguajes nativos (como Kotlin o Swift) o depender de las tiendas de aplicaciones tradicionales para ofrecer un producto instalable, responsivo y con soporte de caché. La web moderna proporciona las herramientas suficientes para emular el comportamiento nativo desde un único código base multiplataforma.

En definitiva, esta práctica superó el objetivo inicial de comprender el funcionamiento del protocolo HTTP en entornos "sin estado", proporcionando una visión holística del ciclo de vida del desarrollo de software actual: desde el control estricto de los datos en la memoria del servidor, hasta la entrega de una experiencia visual inmersiva, interactiva e instalable en el dispositivo del usuario final.